

STORE: Self-Provisioning Storage-as-a-Service for Serverless Functions

<https://arxiv.org/abs/2609.05422>

1. Problem

- 문제: Serverless 함수는 보통 stateless라 외부 Storage/BaaS가 필요한데, 개발자가 스토리지 선택·프로비저닝·권한·연결 설정을 직접 해야 함.
- 왜 중요한가: Serverless가 인프라 관리를 추상화해도 Storage 설정이 남아 있어 개발 복잡성과 설정 오류가 발생함.

2. Method

- 핵심 아이디어: STORE(Self-Provisioning Storage-as-a-Service) — 스토리지 관리를 개발자가 아니라 Serverless 플랫폼이 자동 수행.
- 기존 방법과 차이:
 - Terraform/Pulumi → 개발자가 IaC로 원하는 인프라를 명시해야 함.
 - STORE → 인프라 선언 자체를 제거하고 플랫폼이 runtime에 적절한 Storage를 선택·연결.
- 핵심 메커니즘
 - Dynamic Storage Selection: 데이터 형태/크기/사용 패턴/SLO 등을 보고 적절한 Storage backend 자동 선택.
 - Zero-Touch Configuration: 함수에 Storage endpoint, credential 등을 직접 설정하지 않아도 SDK가 runtime에 자동 해결.
 - Auto-Migration: 기존 Storage API를 STORE 인터페이스로 매핑해 migration 부담 감소.
 - 전체 lifecycle을 Preparation → Resolution → Permission → Selection → Translation으로 분리.

3. Experiments & Results

- 실험 환경: Kubernetes + Knative 기반 prototype, MinIO(파일) + Cassandra(테이블) + Redis.
- 평가: 파일/테이블 저장·조회, sequential workflow, 동시 요청 증가, 실제 serverless use case 등을 평가.
- 개발 effort
 - Terraform/Pulumi 대비 64.6~84.1% 적은 구현량.
 - 예: Shopping Cart workflow → STORE 56 LoC vs Terraform 176 LoC.
- 성능
 - 전체 overhead는 조건에 따라 낮게 유지되며 최저 relative overhead 0.19%.
 - 파일 workload: 평균 overhead 약 71 ms(store), 4.3 ms(retrieve).
 - Table workload: 평균 약 4.6 ms(store), 4.2 ms(retrieve).
 - 요청량 증가에 대해 CPU 사용량이 선형적으로 증가.
- 개선된 것: 성능을 크게 희생하지 않으면서 Storage 관련 개발/설정 작업을 대폭 제거.

4. Insight

- 핵심 기여 한 줄:
 - Serverless에서 Compute뿐 아니라 Storage provisioning까지 플랫폼이 알아서 하게 만들자.
- 한계점
 - 현재 prototype은 binary/tabular data 중심.
 - LoC는 개발 effort의 거친 proxy라 유지보수성·복잡성 전체를 측정하지 못함.
 - Tabular workload에서는 Translation 때문에 overhead가 12.56~31.79%까지 발생.
 - 아직 edge-cloud / multi-cloud 환경까지 확장되지 않음.
- 내가 가져갈 포인트
 - 핵심은 새로운 Storage 자체가 아니라 "Storage 선택/설정이라는 인프라 결정을 application code에서 platform으로 이동"한 것.
 - IaC → Zero-config / Self-provisioning으로 추상화 레벨을 한 단계 높인 접근.

- 특히 serverless에서 “개발자가 무엇을 원하는지만 선언하고, 어디에/어떻게 배치할지는 플랫폼이 결정”하는 방향이 핵심.